



АГЕНТНЫЙ ИИ

Time-Slicing на GPU для параллельных агентов больших языковых моделей в Kubernetes

Перестаньте доверять статусу пода. Комплексная система для измерения скрытых микроархитектурных затрат и конкуренции за память при совместном использовании GPU агентами ИИ в Kubernetes.

Анубхаб Банерджи

14 июня 2026 22 мин на чтение



Обзор архитектуры

Игрушечные агенты запускаются по одному на Python.

Производственные агенты конкурируют за один и тот же графический процессор, и на одной общей карте задержка p99 для агента, чувствительного к задержкам, незаметно увеличилась на 66 %, хотя все модули по-прежнему показывали работоспособность. Вот во что на самом деле обходится эта конкуренция, если измерять ее с помощью p99, а не на глаз.

Это вторая часть серии «Агентный инференс

промышленного уровня». В каждой части из конвейера агентного LLM убирается один вид избыточной работы. В первой части устраняется избыточное предварительное заполнение. Часть 2 (эта часть) посвящена избыточному ожиданию — тому, как несколько микроагентов совместно используют один графический процессор с помощью временной нарезки. Часть 3 обеспечивает поиск RAG на графическом процессоре с помощью пользовательского ядра CUDA Top-K. Часть 4 сохраняет состояние агента при передаче управления, чтобы у следующего агента не возникало проблемы холодного старта.

Основные выводы

- **Совместное использование графического процессора не бесплатно, и ваш планировщик вам об этом не сообщит.** Когда два агента используют один графический процессор с разделением по времени, Kubernetes радостно сообщает, что оба пода имеют `Running`. Проблема кроется в хвосте распределения задержек.

- **Медиана — это ложь, а хвост — правда.** В моем тесте (с двумя агентами) у обоих показатели p50 практически не изменились. Но у маленького, чувствительного к задержкам агента **p99 подскочил с 3,68 мс до 6,10 мс ($\approx 1,66\times$)** и его джиттер (p99/p50) увеличился с **1,02 до 1,70**.
- **Агент, чувствительный к задержкам, деградирует первым.** Маленькая, нестабильная рабочая нагрузка пострадала гораздо сильнее, чем большая и стабильная, хотя у обеих «есть графический процессор».
- **Пропускная способность почти не изменилась, в чем и заключается ловушка.** Пропускная способность прокси-сервера со средней скоростью снизилась всего на несколько процентов, поэтому панель мониторинга, отслеживающая средние показатели, сочла бы это успехом, в то время как ваш агент, чувствительный к редким событиям, спокойно пропустит один дедлайн из пятидесяти.
- **Он работает на графическом процессоре стоимостью 150 долларов.** Everything below is measured on a single five-year-old GTX 1080 with the stock NVIDIA Kubernetes Device Plugin and CUDA time-slicing. No H100, no MIG, no magic. This was intentional, not everyone can afford H100 – some still keep using their old hardware. And honestly, running an agentic AI production on H100 does not require any magic; but on a \$150 GPU, it surely does.

TL;DR: I put two very different agent workloads — a small, latency-sensitive **FFT** worker and a heavy, transformer-style **GEMM** worker — into separate Kubernetes pods, each politely asking for `nvidia.com/gpu: "1"`, and let the NVIDIA device plugin's **CUDA time-slicing** drop them both onto one physical GTX 1080. Then I timed every iteration with CUDA events, rolled it up into p50/p95/p99, computed a degradation factor (shared tail /

solo tail), and cross-checked it against DCGM GPU-utilization counters. *Result:* medians and throughput barely flinched, but **tail latency and jitter blew up — worst for the small, latency-critical agent**. Kubernetes says “two healthy pods.” The silicon says “one of you is starving in the queue.” Kubernetes reports “two healthy pods.” The silicon reports a memory-bus street fight, and the p99 tail tells you who paid the price.

Github Repo:

<https://github.com/AnubhabBanerjee/Kube-Timeslice-Profiler>

(Quick confession before we start: I came at this from a 5G/6G RAN engineering background. As it turns out, it is exactly the kind of problem AI RAN is currently dealing with. On edge servers, operators are trying to co-locate latency-critical baseband processing with heavy LLM inference on the same GPUs. It becomes a scheduling nightmare the second the AI workload starts starving the latency-critical applications of memory bandwidth—and that is exactly why I wrote this post.)

Architecture mental model — keep this open while you read.

Two pods → each asks for `nvidia.com/gpu: 1` → the device plugin cheerfully says "sure, here are 4 GPUs" (there is exactly 1) → CUDA time-slices the one real GPU → everybody takes turns → the tail pays the bill.

Everything below is just commentary on one part of that line.

• • •

1. A confession: “Running” is the most expensive illusion in Kubernetes

Just like the previous post in this series, let us start with a dramatic conversation before we slowly dive into more boring, technical stuff.

You: *“Kubernetes, please run my two agents.”*

Kubernetes: *“Done. Both pods are Running. ”*

You: *“On the same GPU?”*

Kubernetes: *“Yep. Each one asked for `nvidia.com/gpu: 1`, so I gave each one a GPU.”*

You: *“But I only own one GPU.”*

Kubernetes: *“Correct. And I gave each of them a GPU.” *

You: *“Wait, What!? How?? They can’t both have—”*

Kubernetes: *“Shhh. Don’t worry about it. Look how green they are.”*

Your Grafana dashboard: *“Everything looks good, bro. ”*

Meanwhile...

Your physical GPU: *(screaming in context-switches)*

Your p99 latency: *(quietly doubling in the corner)*

Well, maybe it was not that dramatic after all, but you get my point, right? The scheduler's idea of "healthy" is *the pod is alive and a process is running*. It has no opinion about whether your latency-critical agent is getting elbowed off the GPU forty times a second. Pod phase says `Running`. The agent says nothing, because, well, actually nobody asked it.

This follows directly from where [Part 1](#) left off. In the SwarmKV post I had two agents reading one document, and I bragged about prefilling once and fanning the KV cache out. Then, in the caveats, I admitted the embarrassing part: every branch's actual GPU work still ran behind one global mutex. The orchestration fanned out; the *compute* lined up single file. Two agents, two turns. Fifty agents, fifty turns. I hand-rolled a lock and called it a day.

That is fine for a demo. It is a disaster for production, where "an agent swarm" means a dozen small specialized models — a router, a summarizer, a safety checker, a retriever, a pile of tool-callers — all awake at once, all wanting the same accelerator. You cannot buy each of them an H100 (unless your name is Jensen Huang). You pack them onto one shared GPU and *hope the scheduler sorts it out*.

So I wanted to answer one blunt question: **when two agents share one GPU, what does each one actually pay — and will anything in my cluster tell me?**

Spoiler alert: it costs real milliseconds, it lands almost entirely on the small fast agent, and no, nothing in your cluster will tell u. So I built a tool that does.

2. Two agents with opposite personalities

The repo behind this post runs two containerized PyTorch workers that stand in for the two types which you find in basically almost every agent swarm:

- **A small, twitchy, latency-sensitive agent** (`fft_worker.py`). It runs a continuous loop of big 2-D complex FFTs. Think of it as the router / guardrail / tool-caller class — the agents that must answer *now* or the whole world starts falling apart.
- **A big, steady, compute-hungry agent** (`matmul_worker.py`). It runs a continuous stream of large square matrix multiplies — the GEMM at the heart of a transformer forward pass. This is the heavyweight actually doing the model's thinking.

Their entire workload is quite simple for each. The FFT worker pre-allocates a 4096×4096 complex tensor and beats on it:

```
# ----- Pre-allocate tensors -----
# Single allocation keeps cuFFT plan creation and allocator traffi
# ``complex64`` matches typical PHY IQ data width; real-only FFT w
data = torch.randn(MATRIX_SIZE, MATRIX_SIZE, device=device, dtype=
# First launches pay JIT/plan costs; five iterations is a small fi
# Throwaway ``fft2`` calls prime instruction and constant caches s
for _ in range(5):
    # Assignment to ``_`` discards output tensor handle immediatel
    torch.fft.fft2(data)
# Final sync guarantees no warmup kernel overlaps the first timed
sync()
```

The GEMM worker pre-allocates two FP32 matrices and multiplies them forever:

```

# Matmul needs two operands resident on device; allocating once ke
# FP32 is the default training/inference dtype on Pascal-class GPU
A = torch.randn(MATRIX_SIZE, MATRIX_SIZE, device=device)
B = torch.randn(MATRIX_SIZE, MATRIX_SIZE, device=device)
# cuBLAS autotuning can pick different algorithms across first lau
# Five repeats mirror FFT worker so cross-tenant comparisons in pa
for _ in range(5):
    # Result discarded; peak memory stays flat because output tens
    torch.matmul(A, B)
# Sync closes the warmup window so first ``_ev_start.record`` does
sync()

```

The point was never to build a clever model — it was to build two GPU citizens with *opposite manners* and watch them share one room. One finishes in about 3.6 ms and wants to go again immediately; the other takes about 20 ms and just wants to grind. Now put them on the same GPU and ask the only interesting question: *who blinks first?*

Both workers are configured by environment variables, so a pod spec can re-tune them without rebuilding the image:

```

# ----- Configuration (overridable via env vars so pod specs can t
# ``ITERATIONS`` default matches FFT worker so DF numerators/denom
# Raising iterations lengthens shared-GPU ``kubectl wait``; loweri
ITERATIONS = int(os.environ.get("ITERATIONS", 800))
# ``MATRIX_SIZE`` dominates FLOPs per iteration; env override lets
# Time-slicing does not partition memory—both pods' peak allocatio
MATRIX_SIZE = int(os.environ.get("MATRIX_SIZE", 4096))
# ``SLEEP_MS`` defaults slightly above FFT's 100 ms so two tenants
# Same caveat as FFT: sleep is between measured iterations and is
SLEEP_MS = int(os.environ.get("SLEEP_MS", 150))

```

I guess by this point you realize that nothing here is domain specific. The numbers happen to come from a signal-processing workload next to a matmul, but swap in your own two agents — one light and deadline-driven, one heavy and steady — and the

story holds. This is a post about **workload personalities colliding on one accelerator**, not about any one application.

Timing it without being illusioned

There is a classic way to benchmark a GPU and get a beautiful yet completely wrong number: time how long it takes Python to *launch* the kernel. CUDA is asynchronous, so `torch.matmul(A, B)` returns almost instantly while the GPU is still sweating. Measure that and you'll be satisfied that your matmul takes only 50 microseconds, and then you'll start to bang your head wondering why the production is slow.

The workers don't do that. They wrap each operation in CUDA **events** and force a `torch.cuda.synchronize()` so the clock stops *after* the kernels actually retire on the SMs:

```
# Start epoch immediately before ``record`` minimizes gap between
epoch_ns_start = time.time_ns()
_ev_start.record()
_ = torch.fft.fft2(data)
_ev_end.record()
torch.cuda.synchronize()
epoch_ns_end = time.time_ns()
latency_ms_device = float(_ev_start.elapsed_time(_ev_end))
```

`elapsed_time` reads the GPU's own timeline — sub-microsecond resolution, no host-side jitter. That `synchronize()` is the difference between measuring “how long did the GPU work” and “how long did Python take to ask.” Then every iteration coughs up one structured line and flushes it, so Kubernetes log streaming sees it immediately:

```
print(
    f"KTS_APP,v1,FFT,{i},{epoch_ns_start},{epoch_ns_end},{late
)
```

```
# ``flush`` forces line-buffered container stdout through CRI befo  
sys.stdout.flush()
```

Raw silicon execution time goes in; a structured log comes out. A downstream parser aggregates these into exact percentiles, creating a strict measurement contract that strips away all host-side noise.

• • •

3. How two pods end up on one GPU (explained)

This is the part which will feel like magic for people who are new to K8s. For others, you can safely skip this section and move on to the next one.

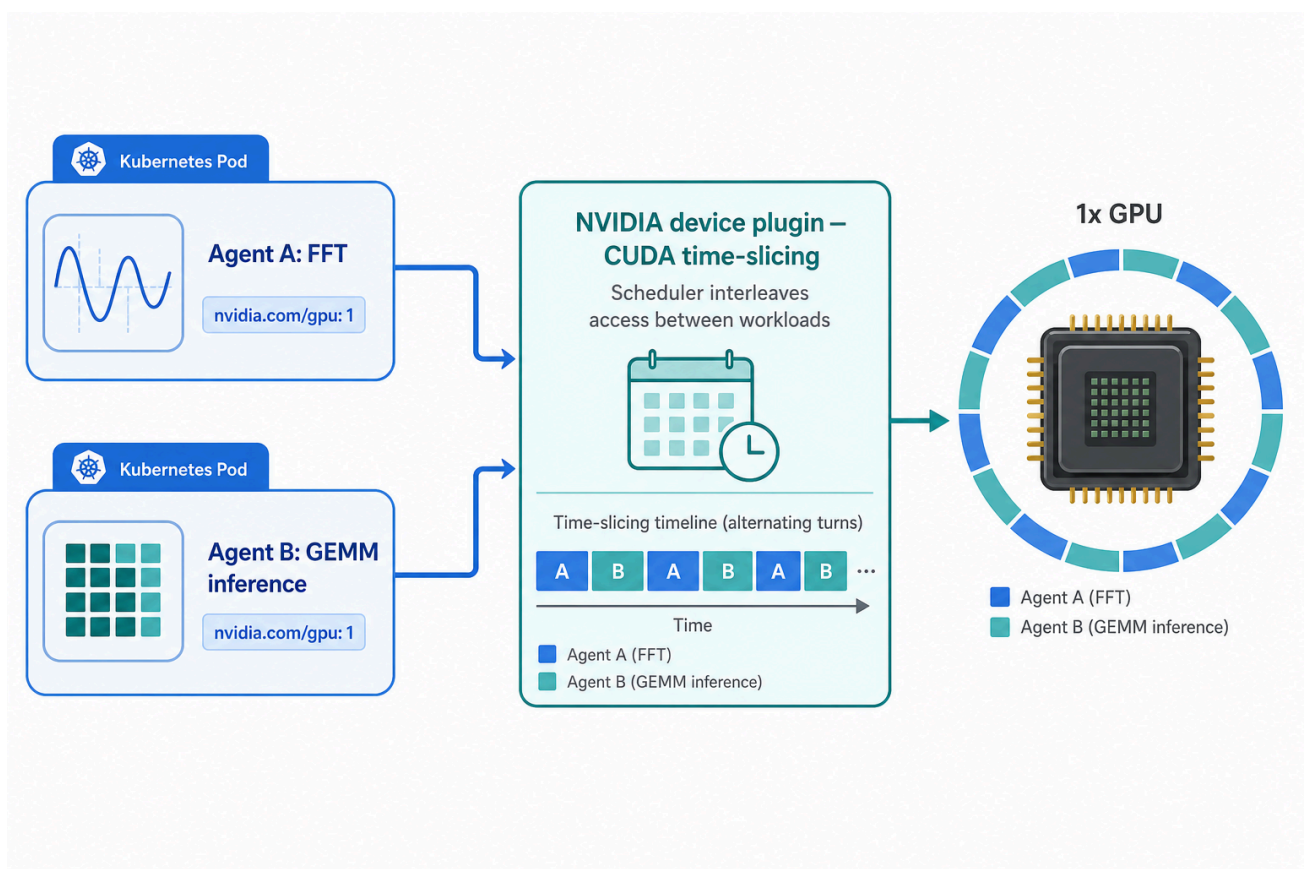
By default, Kubernetes treats `nvidia.com/gpu` as a whole, indivisible thing: one GPU, one claimant, no sharing. The NVIDIA device plugin's **time-slicing** feature changes the *bookkeeping*. You hand it a ConfigMap that says, essentially, “pretend each physical GPU is several”:

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: time-slicing-config  
  namespace: nvidia-device-plugin  
data:  
  any: |-  
    version: v1  
    flags:  
      migStrategy: "none"  
      failOnInitError: true  
    sharing:  
      timeSlicing:  
        failRequestsGreaterThanOne: true  
        renameByDefault: false
```

```
resources:  
- name: nvidia.com/gpu  
  replicas: 4
```

replicas: 4 is Kubernetes for “lie to the scheduler four times.” After this, one physical GTX 1080 advertises **four** allocatable `nvidia.com/gpu` slots to the API. Four pods can each request “1” and all get scheduled, quite happily.

Here is the catch, in bold because the entire post depends on it: **this does not physically partition the hardware.** It is not MIG. There is no memory fence and no compute fence. The four “GPUs” are the same silicon, and the pods take turns on it through CUDA time-slicing — the GPU context-switches between them like a single barista serving four lines by sprinting between registers. More schedulable slots, exactly zero isolation.



The experiment is three Kubernetes Jobs: each agent alone (the baselines), and then both at once. The “both at once” manifest is

the whole ballgame — two Jobs, each innocently asking for one GPU, deliberately landing on the same card:

```
containers:
  - name: worker
    image: localhost/kts-worker:v1
    imagePullPolicy: Never
    resources:
      limits:
        nvidia.com/gpu: "1"
      requests:
        nvidia.com/gpu: "1"
```

Neither pod knows the other exists. Neither asked to share. The scheduler put them in the same room because, as far as it knows, there were four rooms. The baselines tell you how fast each agent runs when it owns the GPU; the shared run tells you what it pays for company. The gap between them is the entire story.

• • •

4. The rig, in one sentence

Everything below runs on a **seven-year-old NVIDIA GTX 1080** (8 GB, Pascal) on a single-node K3s with the stock NVIDIA device plugin and CUDA time-slicing. No H100, no MIG, no datacenter rack — just the card half the people reading this still have under their desk.

I am using this antique on purpose. Bad scheduling doesn't magically vanish on an H100; it just executes its bottlenecks at a higher clock speed. If your agents are fighting over a memory bus on a \$150 card, throwing \$30,000 at the problem won't prevent the traffic jam—it just makes the crash more expensive. Throwing

an H100 at an orchestration flaw doesn't fix the contention; it just lets you execute bad architecture in fewer milliseconds. The physics of cache eviction do not care what year your silicon was minted.

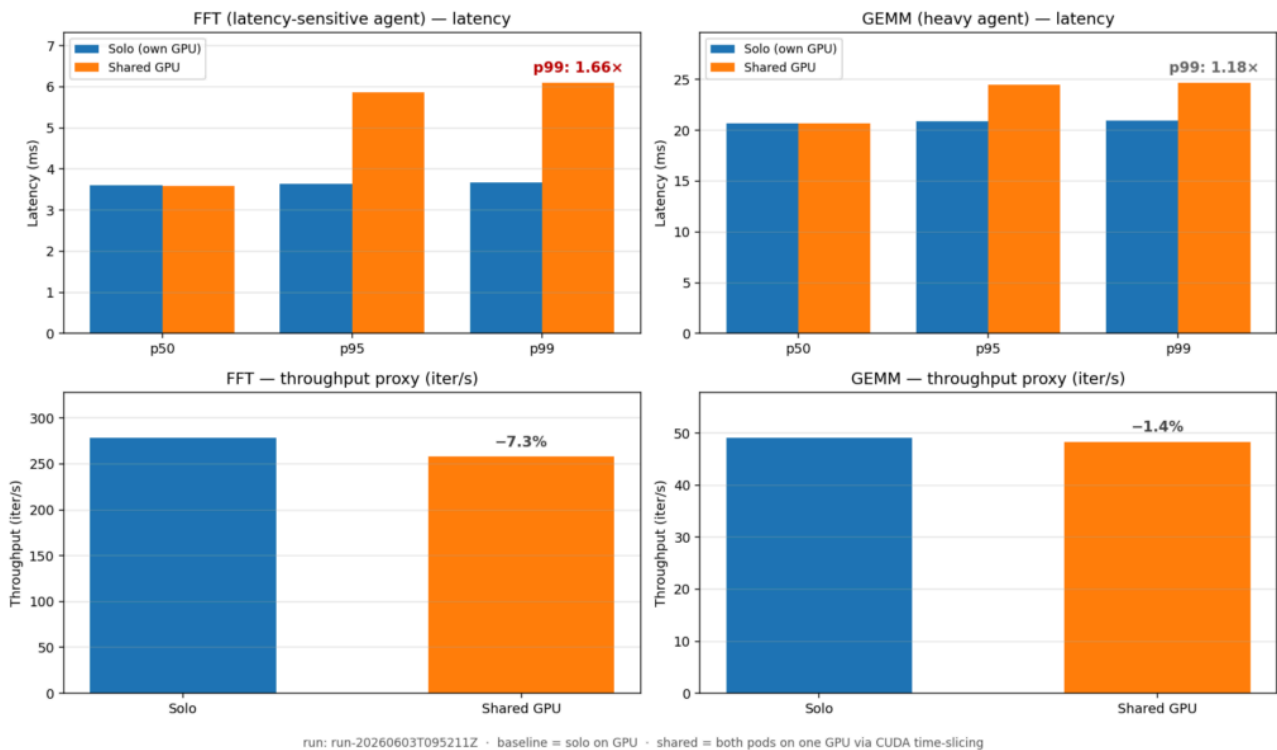
(Driver, containerd, and toolkit versions are pinned in the repo for anyone reproducing this; they are boring on purpose and you do not need them to follow the story.)

• • •

5. The receipts (i.e., the numbers)

Now the whole story in one picture:

Two agents, one time-sliced GTX 1080: medians and throughput barely move; the latency-sensitive agent's tail snaps.



Four panels, one punchline. The medians (the left pair of bars in each latency chart) are basically untouched. The throughputs (bottom row) lost a measly 7.3% and 1.4% — the kind of number u'd report up the chain and get a thumbs-up emoji for. And

then there’s that top-right corner of the top-left chart: **the small agent’s p99 jumped by 66%**. Same dashboard, same Running pods, same boring throughput graph — and one of your two agents is now occasionally, unpredictably 66% slower than it was yesterday. Welcome to GPU sharing.

The actual numbers, so nobody has to squint their eyes at the bars:

Metric	Solo	Shared	Change
FFT (latency-sensitive) p50	3.598 ms	3.593 ms	insignificant
FFT p95	3.645 ms	5.868 ms	1.61×
FFT p99	3.679 ms	6.101 ms	1.66×
FFT jitter (p99/p50)	1.02	1.70	tail blows out
GEMM (heavy) p50	20.677 ms	20.669 ms	insignificant
GEMM p95	20.896 ms	24.505 ms	1.17×
GEMM p99	20.985 ms	24.690 ms	1.18×
GEMM jitter (p99/p50)	1.01	1.20	slight
FFT throughput (iter/s)	278.1	257.9	−7.3%
GEMM throughput (iter/s)	49.1	48.3	−1.4%

Read those FFT rows twice. The **median did not move**. If you were staring at a p50 dashboard you'd swear nothing happened, sign off, and go to lunch. But one in every hundred FFT calls now takes 66% longer, and the gap between a typical iteration and a bad one nearly doubled. You didn't slow the agent down on average — you made it *occasionally, unpredictably late*. Which is worse, because now it's a flaky agent and nobody can reproduce it on a Friday afternoon.

This is the key asymmetry, and it is not a coincidence: **the small, latency-sensitive agent degrades first and worst**. The big GEMM is a bulldozer — it grabs its quantum and grinds through. The little FFT keeps getting tapped on the shoulder mid-stride, shoved off the SMs, and told to wait for its next turn. When two workloads share a single line, the one that needed to be quick is the one that suffers. This has huge implications in the telecom domain: if it keeps happening, calls start to drop and worse comes to worst, even emergency service numbers may also stop functioning. Just let that thought sink in!

To make this comparable across any pair of agents, the tool computes a **degradation factor (DF)** = $\text{shared_p99} / \text{baseline_p99}$. DF = 1.0 means sharing was free. Higher means it hurt. For this run it's **1.66 for the FFT** and **1.18 for the GEMM**. That 1.66 is the entire post compressed into a number you can put on a slide to show to your manager.

And here's the part that should be illegal: **the throughput barely moved**. If your SLO (Service Level Objective) is written in terms of average throughput, you'd look at "FFT down 7%, GEMM down 1%" and declare victory. Meanwhile your tail-sensitive agent is silently missing one deadline in fifty. **Averages are where contention goes**

to hide. The mean is a kind soul who rounds your worst moments away. The p99 is the friend who remembers everything.

One sanity check, then we move on. The profiler also scrapes DCGM GPU-utilization counters every 100 ms and joins them to each iteration. In the shared window, the FFT worker's SM and DRAM activity rise sharply (its execution cycles now overlap with a GEMM hammering the same memory system); in the solo window, they don't. So the contention shows up at two completely independent layers — application latency *and* hardware counters — which is how you know this is real and not a stopwatch artifact.

• • •

6. This is about agent swarms, not any one workload

One could easily label section 5 as “an FFT and a matmul fought over a GPU, which surprises absolutely no one who has ever written a CUDA kernel”, but that misses the point entirely. The two workers are just convenient, measurable stand-ins for a pattern that shows up the instant you put a real agent swarm on shared hardware:

- **The light, deadline-driven agents** — routers, guardrails, classifiers, tool-callers, small fast models. Cheap individually, constantly running, and the whole pipeline waits on them. (The FFT worker is one concrete example of this personality.)
- **The heavy, steady agents** — the big transformer forward passes, the GEMM-bound model calls that dominate

compute. (The GEMM worker is one concrete example of that one.)

Put any two agents with those shapes on one time-sliced GPU and you get exactly what I measured: medians barely twitch, but the **small, latency-critical agent eats the tail**. It does not matter what the agents *do*; it matters how they *behave on the SMs* — one needs to finish fast and often, the other just wants to grind. Time-slicing hands out turns. It does not hand out deadlines. So the agent that lives or dies by its deadline is the one that suffers when its turn keeps getting interrupted.

That is the systems thread running through this whole series. Part 1 was about not *repeating* work across agents (share the KV cache). This part is about not *lying* to yourself about what sharing the GPU costs those agents. Time-slicing buys you **capacity** — more schedulable slots on one card — and gives you zero **isolation**. Watch only averages and your most deadline-sensitive agent breaks first, silently, in the p99, while every pod keeps flashing `Running`.

• • •

7. “So... how do I actually run it?”

The pipeline is deliberately boring, because in systems engineering, ‘exciting’ usually means production is on fire. It’s a linear **build → cluster → logs → metrics** graph driven from the repo root:

1. `run.py` builds the worker image with Podman, imports it into K3s’ containerd, makes the namespace, optionally starts a

DCGM scrape thread, applies the Jobs, waits, and collects logs into `logs/run-<UTC>/`.

2. The workers emit those per-iteration `KTS_APP` lines you saw above.
3. `generate_results.py` parses the logs, trims warmup, computes p50/p95/p99, the throughput proxy, the degradation factor, and the DCGM join, then writes `data/summary.{csv,json}`, the plots, and a `docs/results.md`.

On a node that already has K3s, the NVIDIA driver, the Container Toolkit, the device plugin, and the `nvidia RuntimeClass`, the whole thing is three commands:

```
# 1. Install the time-slicing ConfigMap and reload the device plug  
kubectl apply -f time-slicing-config.yaml
```

```
# 2. Build the worker image and run the full benchmark (build, imp  
python3 run.py
```

```
# 3. Turn the logs into summaries, plots, and a results page  
python3 generate_results.py
```

The repo link? well, you can find it near the top of the article. And congratulations that you made it this far – I hardly thought anyone would ever do!

• • •

8. Honest caveats (because the comments are coming)

This is a small, deliberate study, not a datacenter capacity model. Here is exactly what it is *not*, before someone posts it for me:

1. **It is two agents, not fifty.** The config exposes four logical slots; the highlighted run pairs one FFT worker with one GEMM worker. That's the smallest interesting contention case, picked for clarity. Filling all four slots (the full contention matrix) is on the roadmap, not in these numbers. I am not reporting fifty-agent results, because I did not measure fifty agents.
2. **Throughput is a mean-rate proxy.** $1000 / \text{mean latency}$ is an iteration rate, not request-serving throughput under a real arrival process. It earns its keep for the “averages hide the tail” point and nothing fancier.
3. **The workloads are synthetic.** A looping FFT and a looping matmul are honest stand-ins for a light, latency-sensitive agent and a heavy inference agent, but they are not a fully served model behind real traffic. The *interference shape* generalizes; the absolute milliseconds do not.
4. **DCGM activity is a low-magnitude proxy.** The workers pace themselves with sleeps, so the GPU idles a lot and the SM/DRAM means look small. Treat them as relative, within-study signals — they corroborate the latency story, they don't claim full saturation.
5. **Time-slicing is not the only sharing mode.** As §7 lays out, this study deliberately measures the default path — the one most people get the moment they flip on GPU sharing. A head-to-head with MPS and MIG is a separate post.
6. **One GPU class, one run highlighted.** Numbers come from a single Pascal GTX 1080. Newer GPUs context-switch faster and the absolute tails shrink; the *direction* — small latency-sensitive agent degrades first — is the durable result.

None of this moves the takeaway. It just keeps me honest about its scope — and the moment a benchmark post hides its caveats is the moment its numbers stop being worth anything.

• • •

9. Wrap (and the setup for Part 3)

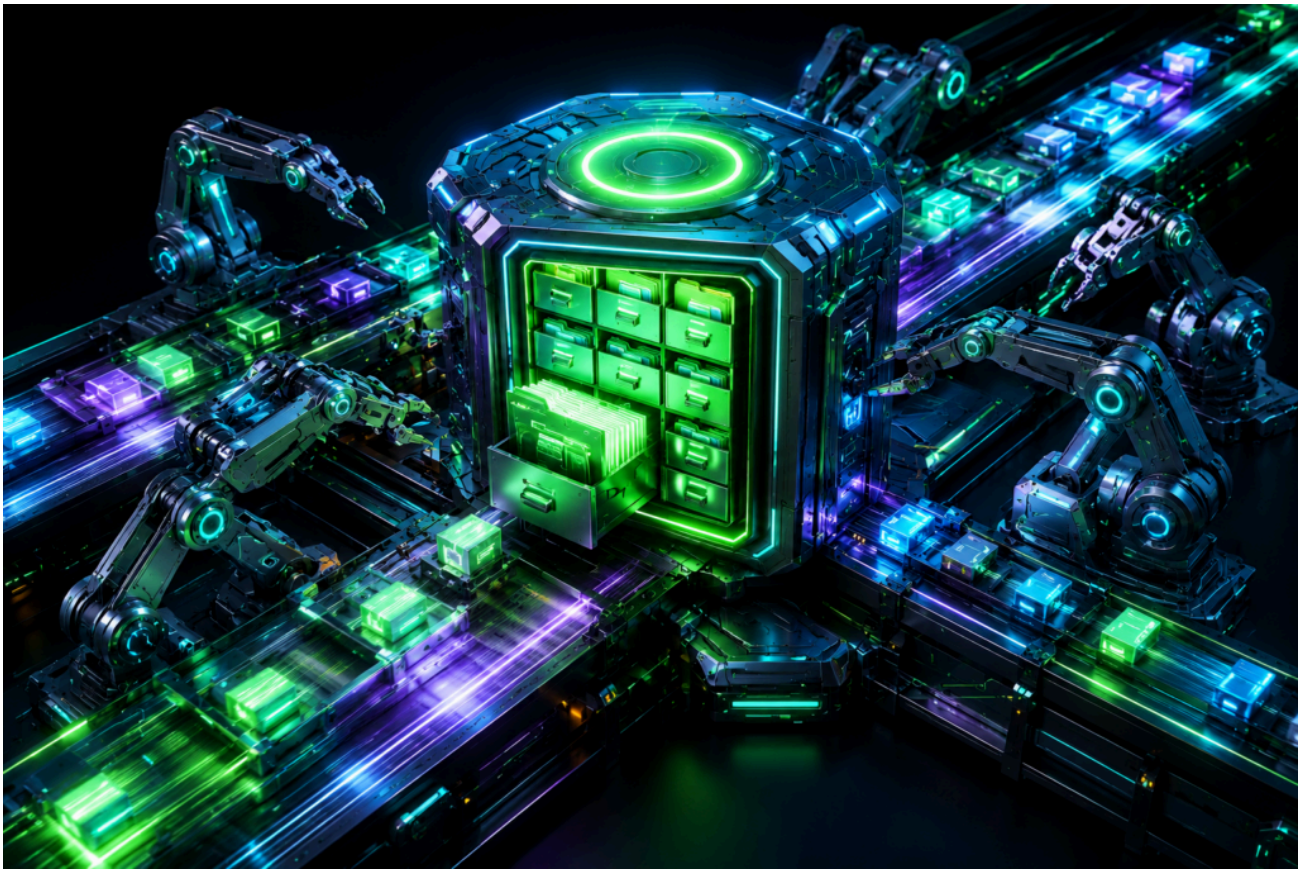
Kubernetes time-slicing is a wonderful illusion. It tells your scheduler that one GPU is four, lets four pods report `Running`, and then quietly locks them in a room to fight over the memory bus. For throughput-bound, deadline-relaxed work, that illusion is harmless and genuinely useful. For the latency-sensitive members of an agent swarm, the illusion hides exactly where you are not looking: the p99.

The solution isn't to ban GPU sharing—you have to share hardware, unless you have an infinite budget. The solution is to stop using a green YAML checkmark as a substitute for microarchitectural reality. Measure the tail, attribute the degradation, and schedule with actual silicon limits in mind.

`Kube-TimeSlice-Profiler` is a step towards the right direction: it turns the vague feeling of “the GPU seems slow today” into a measurable Degradation Factor with receipts.

If you came here as a beginner who just wanted to know why “both pods are `Running`” doesn't mean “both agents are happy”: congratulations, you now understand GPU sharing better than the green checkmark does. Go ahead and distrust your averages, you are ready!

Coming up next: The PCIe Walk of Shame



We just survived two agents fighting over a single GPU without lying to ourselves about the latency tail. But there's another silent tax buried in every RAG pipeline: the PCIe commute.

Right now, every time an agent needs to retrieve context, it pauses, leaves the accelerator, crawls across the PCIe bus back to Python, runs a vector search on the CPU, and trudges all the way back.

In Part 3, we are killing that commute. We will build a custom **CUDA Top-K kernel** to keep the entire retrieval loop trapped on the GPU hardware—no Python round-trips, no host-side delays. Same budget GPU. Same “stop wasting hardware” philosophy.

See you in Part 3.

• • •

Disclaimer: The illustrations in this article were generated using AI (Claude Opus 4.8). They are illustrative, not photographic, and any labels visible inside the images are stylized rather than

authoritative — refer to the article body and the code itself for precise function names, metric values, and architecture details.

• • •

WRITTEN BY

Anubhab Banerjee

See all from Anubhab Banerjee

Agentic Ai

Deep Dives

Kubernetes

Mlops

system architecture

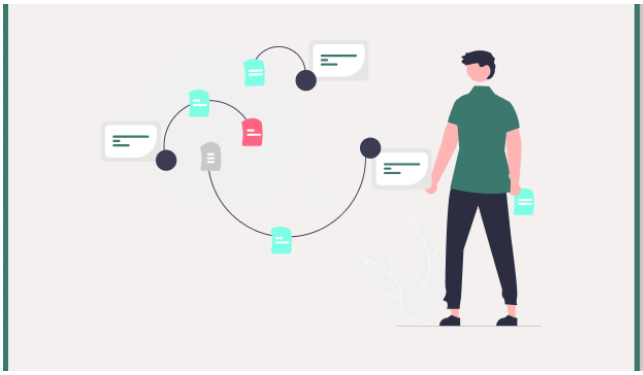
Share This Article



Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

Related Articles



AGENTIC AI

LangChain for EDA: Build a CSV Sanity-Check Agent in Python

A practical LangChain tutorial for data scientists to inspect CSVs

Sarah Schürch

September 9, 2025 19 min read



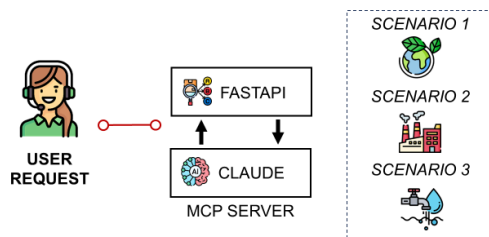
AGENTIC AI

The Rise of Semantic Entity Resolution

Semantic entity resolution uses language models to bring an increased level of automation to schema...

Russell Journey

September 14, 2025 22 min read



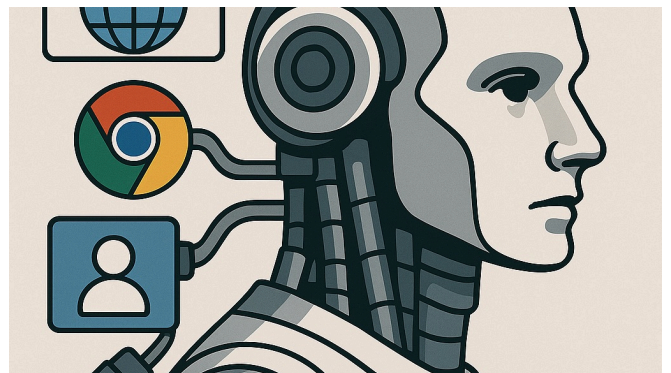
AGENTIC AI

How to Connect an MCP Server for an AI-Powered, Supply-Chain Network Optimization Agent

From prompt to strategic decision-making: MCP-powered agents for cost-efficient, reliable and sustainable supply chain network...

Samir Saci

September 22, 2025 22 min read



AGENTIC AI

MCP in Practice

Mapping power, concentration, and usage in the emerging AI developer ecosystem

Sruly Rosenblat

September 29, 2025 14 min read



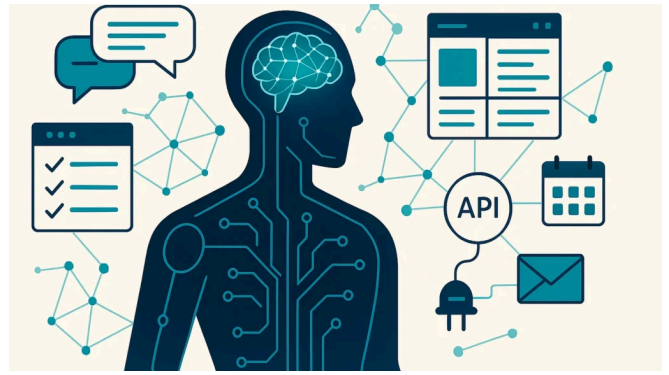
AGENTIC AI

How to Build Effective Agentic Systems with LangGraph

Create AI workflows with agentic frameworks

Eivind Kjosbakken

September 30, 2025 12 min read



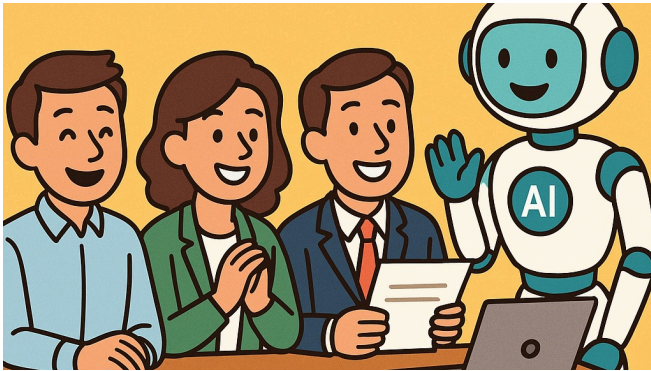
AGENTIC AI

How to Build An AI Agent with Function Calling and GPT-5

How an AI agent works: a step-by-step guide

Ayoola Olafenwa

October 20, 2025 15 min read



AGENTIC AI

How to Automate Workflows with AI

Learn how to take a manual process and optimize it using AI

Eivind Kjosbakken

November 15, 2025 7 min read



Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2026

[Subscribe to Our Newsletter](#)

[WRITE FOR TDS](#) · [ABOUT](#) · [ADVERTISE](#) · [PRIVACY POLICY](#) · [TERMS OF USE](#)